

Show command palette (Ctrl+Shift+P)

```
# =====  
# Student Name : Srushti Kale  
# PRN          : 23070521067  
# Classification using Deep Feed Forward Network  
# =====  
  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
import tensorflow as tf  
from tensorflow import keras  
from sklearn.datasets import load_breast_cancer  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import StandardScaler  
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report  
  
# --- 1. Load Dataset ---  
# Srushti Kale - 23070521067: Loading the Breast Cancer classification dataset  
data = load_breast_cancer()  
X = data.data  
y = data.target  
  
print("Dataset Shape:", X.shape)  
print("Target Classes:", data.target_names)  
  
# --- 2. Data Preprocessing ---  
# Srushti Kale - 23070521067: Splitting data into 80% training and 20% testing  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)  
  
# Srushti Kale - 23070521067: Standardizing the features for the neural network  
scaler = StandardScaler()  
X_train_scaled = scaler.fit_transform(X_train)  
X_test_scaled = scaler.transform(X_test)  
  
# --- 3. Design Deep Feed Forward Network (DFFN) ---  
# Srushti Kale - 23070521067: Building a binary classification model  
model = keras.Sequential([  
    keras.layers.Dense(64, activation='relu', input_shape=(X_train_scaled.shape[1],)),  
    keras.layers.Dense(32, activation='relu', name="Hidden_2"),  
    keras.layers.Dense(1, activation='sigmoid', name="Output_Layer") # Sigmoid  
)  
  
model.summary()  
  
# --- 4. Model Compilation ---  
# Srushti Kale - 23070521067: Compiling with Adam optimizer and binary cross entropy loss  
model.compile(  
    optimizer='adam',  
    loss='binary_crossentropy',  
    metrics=['accuracy']  
)
```

```

# --- 5. Model Training ---
# Srushti Kale - 23070521067: Training the classifier
history = model.fit(
    X_train_scaled,
    y_train,
    batch_size=16,
    validation_split=0.2,
    verbose=1
)

# --- 6. Model Testing and Evaluation ---
# Srushti Kale - 23070521067: Predicting on the test set
# Convert probabilities to binary class labels (0 or 1) using a 0.5 threshold
y_pred_probs = model.predict(X_test_scaled)
y_pred = (y_pred_probs > 0.5).astype("int32")

# Calculate Accuracy
acc = accuracy_score(y_test, y_pred)
print(f"\n--- Testing Accuracy (Srushti Kale - 23070521067) ---")
print(f"Accuracy: {acc * 100:.2f}%")
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=data.target_names))

# --- 7. Analyze and Visualize Confusion Matrix ---
# Srushti Kale - 23070521067: Generating the confusion matrix
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=data.target_names, yticklabels=data.target_names)
plt.title('Confusion Matrix - Srushti Kale (23070521067)')
plt.ylabel('Actual Label')
plt.xlabel('Predicted Label')
plt.show()

```

Show command palette (Ctrl+Shift+P)

Show command palette (Ctrl+Shift+P)

Dataset Shape: (569, 30)
 Target Classes: ['malignant' 'benign']
 /usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106: U
 super().__init__(activity_regularizer=activity_regularizer, **kwargs)
 Model: "sequential"

Show command palette (Ctrl+Shift+P)

	Output Shape	Param #
Hidden_1 (Dense)	(None, 64)	1,984
Hidden_2 (Dense)	(None, 32)	2,080
Output_Layer (Dense)	(None, 1)	33

Total params: 4,097 (16.00 KB)
 Trainable params: 4,097 (16.00 KB)
 Non-trainable params: 0 (0.00 B)

Epoch 1/50
 23/23 ————— 4s 37ms/step - accuracy: 0.8681 - loss: 0.4095 - v
 Epoch 2/50
 23/23 ————— 0s 13ms/step - accuracy: 0.9451 - loss: 0.1870 - v
 Epoch 3/50
 23/23 ————— 1s 16ms/step - accuracy: 0.9670 - loss: 0.1213 - v
 Epoch 4/50
 23/23 ————— 1s 26ms/step - accuracy: 0.9780 - loss: 0.0926 - v
 Epoch 5/50
 23/23 ————— 1s 28ms/step - accuracy: 0.9835 - loss: 0.0768 - v
 Epoch 6/50
 23/23 ————— 1s 29ms/step - accuracy: 0.9835 - loss: 0.0655 - v
 Epoch 7/50
 23/23 ————— 1s 22ms/step - accuracy: 0.9863 - loss: 0.0583 - v
 Epoch 8/50
 23/23 ————— 0s 20ms/step - accuracy: 0.9863 - loss: 0.0512 - v
 Epoch 9/50
 23/23 ————— 1s 15ms/step - accuracy: 0.9863 - loss: 0.0468 - v
 Epoch 10/50
 23/23 ————— 0s 10ms/step - accuracy: 0.9890 - loss: 0.0431 - v
 Epoch 11/50
 23/23 ————— 0s 5ms/step - accuracy: 0.9918 - loss: 0.0374 - va
 Epoch 12/50
 23/23 ————— 0s 5ms/step - accuracy: 0.9890 - loss: 0.0350 - va
 Epoch 13/50
 23/23 ————— 0s 5ms/step - accuracy: 0.9890 - loss: 0.0316 - va
 Epoch 14/50
 23/23 ————— 0s 5ms/step - accuracy: 0.9918 - loss: 0.0278 - va
 Epoch 15/50
 23/23 ————— 0s 6ms/step - accuracy: 0.9945 - loss: 0.0260 - va
 Epoch 16/50
 23/23 ————— 0s 5ms/step - accuracy: 0.9945 - loss: 0.0225 - va
 Epoch 17/50
 23/23 ————— 0s 5ms/step - accuracy: 0.9945 - loss: 0.0207 - va
 Epoch 18/50
 23/23 ————— 0s 5ms/step - accuracy: 0.9945 - loss: 0.0188 - va
 Epoch 19/50
 23/23 ————— 0s 5ms/step - accuracy: 0.9945 - loss: 0.0176 - va
 Epoch 20/50
 23/23 ————— 0s 5ms/step - accuracy: 0.9973 - loss: 0.0152 - va
 Epoch 21/50
 23/23 ————— 0s 5ms/step - accuracy: 0.9973 - loss: 0.0135 - va
 Epoch 22/50